

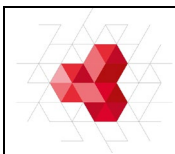
ACTIVIDADES DE SCRIPTING

Script de Bash para practicar:

Hasta el punto **15** se harán en un solo archivo “**nombrebash.sh**” con un menú que según la opción elegida ejecute el script deseado, hay que tener en cuenta que cuando se habla de parámetros se han de pedir los datos al usuario en el script principal y pasar esos valores como parámetros a la llamada de la función, recordar subir el trabajo diario al repositorio del módulo.

1. Script “**bisiesto**” que devuelve si el año pasado como **parámetro** es bisiesto.
2. Script “**configurarred**” que modifica la configuración de red del equipo y a continuación la muestre, los datos necesarios se pasarán como **parámetro**. (IP- MÁSCARA – PUERTA DE ENLACE - DNS)
3. Script “**adivina**”, juego donde el sistema crea un número aleatorio entre 1 y 100 y el usuario tiene que adivinarlo, para ellos solo tiene 5 oportunidades. En cada intento se le mostrará información al jugador del número de intentos y de si el valor que ha introducido es menor o mayor del número a adivinar. Al final del juego se le mostrará el número de intentos realizados, o en caso de no conseguirlo se le informará que ya no tiene más intentos y se le mostrará el número aleatorio.
4. Script “**buscar**” que solicita al usuario el nombre de un fichero y se buscará en el sistema. En caso de que el fichero no exista devolverá un mensaje de error, y si el fichero existiese mostrará el directorio donde se encuentra y el número de vocales que contiene dicho archivo.
5. Script “**contar**” que diga cuantos ficheros hay en un directorio que se pide por teclado.
6. Script “**permisosoctal**” dado un nombre de objeto mostrar los permisos de dicho objeto en octal, se han de tener en cuenta los permisos especiales. (ruta absoluta y suponemos que existe),
7. Script “**romano**” Script que solicite un número de 1 al 200 al usuario y muestre su representación en romano. (I=1, V=5, X=10, L=50, C=100).
8. Script “**automatizar**” que compruebe si hay elementos en el directorio `/mnt/usuarios` si este está vacío mostrará un mensaje de listado vacío y en caso afirmativo creará tantos usuarios como documentos, con el mismo nombre del documento. Además, dentro del documento en cada línea se encontrará el nombre de las carpetas que se han de crear en el directorio personal del usuario. Una vez creado el usuario y las carpetas solicitadas borrará el archivo del directorio.
9. Script “**crear**” que admita dos parámetros, el primero indicará el nombre de un fichero, y el segundo su tamaño. El script creará en el directorio actual un fichero con el nombre dado y el tamaño dado en Kilobytes. En caso de que no se le pase el segundo parámetro, creará un fichero con 1.024 Kilobytes y el nombre dado. En caso de que no se le pase ningún parámetro, creará un fichero con nombre `fichero_vacio` y un tamaño de 1.024 Kilobytes. Ejemplo:

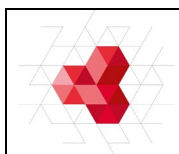
- crear.sh aguado 546 (creará el fichero aguado con 546 K de tamaño).
 - crear.sh panadero (creará el fichero panadero con 1.024 K de tamaño).
 - crear.sh (creará el fichero fichero_vacio con 1.024 K de tamaño).
10. Script “**crear_2**” similar al anterior, modificando para que antes de crear el fichero compruebe que no exista. En caso de que exista avisará del hecho por pantalla y creará el fichero, pero añadiéndole un 1 al final del nombre (aguado1, por ejemplo). Si también existe un fichero con ese nombre, lo creará con un 2 al final del nombre, así seguiremos hasta intentar el 9. Si también existe un fichero con 9 al final del nombre, avisará del hecho y no creará nada.
 11. Script “**reescribir**” que acepte como parámetro una palabra. El script debe reescribir la palabra por la pantalla, pero cambiando la a por un 1, la e por un 2, la i por un 3, lo o por un 4 y la u por un 5.
 12. Script “**contusu**” que nos diga por pantalla cuantos usuarios reales tiene nuestro sistema (usuarios que tengan un directorio creado en /home), nos deje elegir de una lista el nombre de uno de ellos, y le realice automáticamente una copia de seguridad de todo su directorio home en /home/copiasseguridad/nombreusuario_fecha. Nombreusuario será el nombre del usuario, y _fecha será la fecha actual del sistema. Nos referimos a usuarios normales que tengan creado una carpeta en /home.
 13. Script “**quita_blanco**” que automáticamente debe, renombrar todos los ficheros del directorio actual de modo que se cambien todos los espacios en blanco de los nombres de los ficheros por subrayados bajos (el carácter _). Así, si en el directorio indicado como parámetro hay un fichero como Mis mejores juegos al ejecutar el script cambiará su nombre por Mis_mejores_juegos. Esto debe hacerse automáticamente para todos los ficheros del directorio actual que tengan espacios en blanco en el nombre.
 14. Script “**lineas**” que aceptará tres parámetros, el primero será un carácter cualquiera, el segundo un número entre 1 y 60 y el tercero un número entre 1 y 10. El script debe dibujar por pantalla tantas líneas como indique el parámetro 3, cada línea formada por tantos caracteres del tipo parámetro 1 como indique el número indicado en parámetro 2. El script debe controlar que no se le pase alguno de los parámetros y que los números no estén comprendidos entre los límites indicados. Ejemplo: ./lineas.sh k 20 5 (escribe 5 líneas, cada una formadas por 20 letras k).
 15. Script “**analizar**” que analizará el número y tipo de documentos en el árbol de directorios especificado como **parámetro**. Además, se pasarán por argumentos las extensiones que queramos analizar. Por ejemplo: *analizar.sh /home/yeray/www jpg gif html txt* me hará un informe del directorio y todos sus subdirectorios según el tipo de archivos (cuantos son).



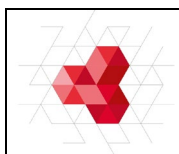
Script de PowerShell para practicar:

Hasta el punto **30** se harán en un solo archivo “**nombrepower.ps1**” con un menú que según la opción elegida ejecute el script deseado, hay que tener en cuenta que cuando se habla de parámetros se han de pedir los datos al usuario en el script principal y pasar esos valores como parámetros a la llamada de la función, recordar subir el trabajo diario al repositorio del módulo.

16. Script “**pizza**” La pizzería Bella Napoli ofrece pizzas vegetarianas y no vegetarianas a sus clientes. Los ingredientes para cada tipo de pizza son: vegetarianos: Pimiento y tofu y **no** vegetarianos: Pepperoni, Jamón y Salmón. Escribir script que pregunte al usuario si quiere una pizza vegetariana o no, y en función de su respuesta le muestre un menú con los ingredientes disponibles para que elija. Solo se puede elegir un ingrediente además de la mozzarella y el tomate que están en todas las pizzas. Al final se debe mostrar por pantalla si la pizza elegida es vegetariana o no y todos los ingredientes que lleva.
17. Script “**días**” Calcular el número de días pares e impares que hay en un año bisiesto. ***Pares 179 e impares 187.***
18. Script “**menu_usuarios**” que haga las siguientes acciones:
 - Listar usuarios
 - Crear usuarios (pide usuario y contraseña)
 - Elimina usuarios (pide usuario)
 - Modifica usuarios (pide usuario y nuevo nombre)
19. Script “**menu_grupos**” que haga las siguientes acciones:
 - Listar grupos y miembros
 - Crear grupo (pide nombre grupo)
 - Elimina grupo (pide nombre grupo)
 - Crea miembro de un grupo (pide grupo y usuario)
 - Elimina miembro de un grupo (pide grupo y usuario)
20. Script “**diskp**”, pedirá al usuario por consola el número del disco a utilizar, devolverá tu tamaño en GB, y con la utilización de la herramienta **Diskpart** limpiará el disco y creará particiones de 1GB hasta que no quede espacio en el disco.
21. Script “**contraseña**”, Crear un script en powershell que compruebe la validez de una contraseña. Para ser válida, deberá contener:
 - letras minúsculas
 - letras mayúsculas
 - números
 - caracteres especiales
 - tener al menos 8 caracteres
22. Script “**Fibonacci**”, imprimir los n primeros números de Fibonacci.



23. Script “**Fibonacci**”, imprimir los n primeros números de Fibonacci, para ello se ha de usar la recursividad.
24. Script “**monitoreo**”, monitoree el uso de CPU durante 30 segundos (tomando una medida cada 5 segundos) y al final muestre el promedio de uso.
25. Script “**alertaEspacio**”, revise las unidades del sistema y, si alguna tiene menos del 10% de espacio libre, genere un aviso en pantalla y escriba la alerta en un fichero de log.
26. Script “**copiasMasivas**”, recorra cada usuario con carpeta en C:\Users y haga una copia comprimida de su perfil en C:\CopiasSeguridad\nombreusuario.zip.
27. Script “**automatizarps**” que compruebe si hay elementos en el directorio *C:\usuarios* si este está vacío mostrará un mensaje de listado vacío y en caso afirmativo creará tantos usuarios como documentos, con el mismo nombre del documento. Además, dentro del documento en cada línea se encontrará el nombre de las carpetas que se han de crear en el directorio personal del usuario. Una vez creado el usuario y las carpetas solicitadas borrará el archivo del directorio.
28. Script “**evento**” Desarrolla un script que recopile los últimos 200 eventos críticos o de error del Visor de Eventos (System y Application) y los exporte a un fichero CSV. El script debe permitir pasar como parámetro el número de eventos a extraer.
29. Script “**Agenda**” Se guardan nombres y números de teléfono en una agenda, existirá un menú con las siguientes opciones: (Implementar el script con un diccionario)
- ☐ **Añadir/modificar**: Nos pide un nombre. Si el nombre se encuentra en la agenda, debe mostrar el teléfono y, opcionalmente, permitir modificarlo si no es correcto. Si el nombre no se encuentra, debe permitir ingresar el teléfono correspondiente.
 - ☐ **Buscar**: Nos pide una cadena de caracteres, y nos muestras todos los contactos cuyos nombres comiencen por dicha cadena.
 - ☐ **Borrar**: Nos pide un nombre y si existe nos preguntará si queremos borrarlo de la agenda.
 - ☐ **Listar**: Nos muestra todos los contactos de la agenda.
30. Script “**limpieza**” que utilice *param()* para recibir parámetros desde la consola. El script debe aceptar tres parámetros obligatorios: **-Ruta**, que indica la carpeta donde se realizará la limpieza; **-Dias**, que especifica cuántos días deben tener los archivos para ser eliminados si son más antiguos; y **-Log**, que define la ruta del fichero donde se registrarán las acciones. Además, debe incluir un parámetro opcional **-WhatIf** que permita simular la ejecución sin borrar realmente los archivos, mostrando únicamente cuáles se eliminarían. El script debe comprobar que la carpeta indicada existe, y en caso contrario mostrar un error. Si la carpeta es válida, debe eliminar los archivos que cumplan la condición, mostrando en pantalla el nombre y fecha de los que se borran y registrando en el log la fecha, el archivo eliminado y su tamaño. Por ejemplo, al ejecutar **.\limpieza.ps1 -Ruta "C:\Temp" -Dias 30 -Log "C:\reportes\limpieza.log" -WhatIf**, el script debería mostrar qué archivos serían eliminados sin llegar a borrarlos realmente.



Scripts individuales

31. Script “nombre01” (bash)

El departamento de informática de un Centro de Enseñanza Artística dispone de un servidor Linux que gestiona varios repositorios de ficheros de imágenes que los alumnos utilizan, según la materia en la que estos están matriculados. El administrador del sistema, desea preparar un script para la shell bash, que le facilite el trabajo de mantenimiento de dichos repositorios.

Se dispone actualmente de 3 repositorios denominados **Fotografía**, **Dibujo** e **Imágenes**. Estos repositorios almacenan ficheros con 3 extensiones posibles o válidas, que son jpg, gif y png. Estos ficheros son alojados por parte del alumnado matriculado en las materias (materias cuyo nombre coincide con el nombre del repositorio).

Se puede identificar la procedencia de cada fichero mediante su propietario y grupo. Así, por ejemplo, si se realiza un listado de alguno de esos repositorios aparecería la información de los siguientes campos:

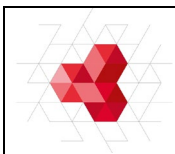
```
<permisosdelfichero><propietario><grupo><tamaño><mes><dia><hora><nombredelfichero>  
rw- r- - r- - alumnoluis fotografia 35000 mar 15 22:30 foto1.jpg  
rw- r- - r- - alumnofran fotografia 45000 abr 25 11:30 fotofinal.png  
rw- r- - r- - alumnoluis fotografia 55000 abr 26 22:30 icono.gif
```

Este script debe comprobar, para los tres repositorios, si cada fichero indicado con una de estas 3 extensiones está en el formato correcto. Para ello se hará uso del comando **file** que con la opción **-i** devuelve el formato que reconoce el sistema independiente de la extensión del archivo.

Si la extensión del fichero difiere de las extensiones que son válidas, pero su formato interno coincide con alguna de ellas, debe renombrarse su extensión al formato correcto. Si el fichero no tiene el formato interno deseado, éste debe eliminarse del sistema, dejando registro del borrado en un fichero llamado descartados.log. Un ejemplo del contenido de este fichero sería:

```
<propietario>;<grupo>;<fechadeborrado>;<nombredelfichero>  
alumnoluis;fotografia;23.06.2018; fotofinal.jpg
```

Además, se desea que el script se comporte de forma diferente al pasarle un parámetro (el nombre del alumno) y compruebe, en este caso, el total de ficheros borrados para ese alumno.



32. Script “nombre02” (bash)

La empresa T-SAI SL que trabaja por proyectos, necesita dar de alta a los programadores contratados como usuarios en su servidor Ubuntu Server. Cuando el proyecto finaliza, la empresa les tiene que dar de baja del servidor. Se pide realizar **un script** bash que se encargue de dar de baja del sistema a dichos usuarios, teniendo en cuenta lo siguiente:

1. El script debe recibir un fichero por parámetro (por ejemplo, el fichero bajas.txt). Dicho fichero contendrá una fila por cada usuario/a a dar de baja.
2. Cada fila del fichero que se pasa por parámetro contiene la siguiente estructura:

nombre:apellido1:apellido2:login

Ejemplo del fichero bajas.txt:

```
Luisa maria:rodríguez:santana:lmrodsan
Juan:Suárez:Solana:jsuasol
Daniel:Amador:Moedano:damamoe
```

Se deben realizar las siguientes comprobaciones, generando los mensajes de error correspondientes por pantalla:

- Que se pasa únicamente un parámetro.
 - Que el parámetro es un fichero y existe.
3. Por cada usuario/a se debe además hacer lo siguiente:
 - Verificar que el usuario existe en el sistema, si no existe, se añade una línea al fichero **bajaserror.log** en el directorio de log del sistema.
 - La estructura de cada línea del fichero tendrá la siguiente estructura:
fecha-hora-login-nombre-apellidos-motivo_de_error

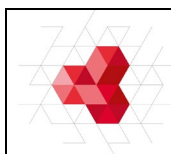
```
19/06/2021-09:05:21-damamoe-daniel-amador-moedano-ERROR:login no existe en el sistema
19/06/2021-10:14:36-ppinher-pilar-del pino-hernández-ERROR:login no existe en el sistema
(ejemplo fichero bajaserror.log)
```

4. Se creará una carpeta con el login del usuario/a a eliminar dentro de /home/proyecto/ y se moverá a ella solo los ficheros del directorio **trabajo** que se encuentra en el directorio personal del usuario correspondiente.
5. Se debe registrar en otro fichero de log (bajas.log), la fecha y hora, el login del usuario, la carpeta a la que se mueven los ficheros, el listado numerado de todos los ficheros movidos, indicando al final el total de ficheros.

Ejemplo bajas.log

```
19/06/2021-09:05:21-lrodsan-/home/proyecto/lrodsan
1:proyecto1.java
2:estilos.css
3:formulario.html
4:proyecto2.js
Total de ficheros movidos: 4
```

6. El nuevo propietario de los ficheros será root
7. Se borrará cada usuario/a del sistema, así como todos sus ficheros y directorios personales.

**33. Script “nombre03” (powershell)**

La empresa **T-SAI SL** que trabaja por proyectos, necesita realizar un fichero en powershell para su servidor Windows Server. La tarea consiste en automatizar el cambio de determinados usuarios de departamento modificando el ActiveDirectory.

La ruta de archivos y el fichero con los usuarios se pasan por **parámetros**:

cambiousuarios.ps1 -ruta c:\usuarios\yeray\Documentos -fichero Cambio_de_departamento.csv

El formato del fichero .csv (con delimitador estándar) llamado Cambio_de_departamento.csv es el siguiente:

	A	B	C	D	E	F	G	H	I	J
1	sAMAccountName	businessCategory	company	department	employeeID	employeeType	physicalDeliveryOfficeName	st	streetAddress	wwwHomePage
2	11111111D	Administración	ETT	Administración	11111	externo	11111XM	España	C.\ Triana	Encargado
3	22222222B	Almacen	E. Servicio	Almacen	22222	externo	11111XP	España	C.\ Leon	Empleado
4	33333333c	Taller	La Mia	Taller	33333	Interno	11111ZP	España	C.\ Castillo	Mecanico
5										

- Se debe comprobar que el usuario exista en el Directorio Activo.
- En caso de no existir se generará un fichero de salida **Errores.csv** con el siguiente formato:

	A	B	C
1	Fecha	sAMAccountName	Error
2	20/06/2021 14:04	11111111D	Usuario erróneo o no existe
3	20/06/2021 14:06	22222222B	Usuario erróneo o no existe
4	20/06/2021 14:06	33333333c	Usuario erróneo o no existe
5			

- Se debe generar un fichero de salida **Resultado.csv** con los usuarios modificados que contendrá los mismos campos que el fichero de entrada.